

Algorytm na przetwarzanie rozproszone

Temat: Taras widokowy

1 Parametry

- N - liczba procesów, stała
- W - wielkość windy, stała
- T - wielkość tarasu, stała

2 Struktury i zmienne

- `clock` - zegar Lamporta, początkowo 0
- `state` - aktualny stan procesu
- `queue_taras` - lokalna kolejka żądań o taras, posortowana rosnąco po (timestamp, PID)
- `queue_winda_dol` - lokalna kolejka żądań o poczekalnię na dole
- `queue_winda_gora` - lokalna kolejka żądań o poczekalnię na górze
- `ack_taras` - licznik ACK dla bieżącego żądania o taras
- `ack_winda_gora` - licznik ACK dla poczekalni na górze
- `ack_winda_dol` - licznik ACK dla poczekalni na dole
- `queue_request` - lokalna kolejka procesów z mniejszym priorytetem, które ubiegają się o ten sam stan co lokalny proces

3 Wiadomości

Wszystkie wiadomości są posiadają znacznik czasowy (timestamp), modyfikowany zgodnie z zasadami skalarnego zegara logicznego Lamporta.

1. `REQ_TARAS` - żądanie ubiegania się o miejsce na tarasie; zawiera priorytet żądania (timestamp, PID)
2. `ACK_TARAS` - potwierdzenie żądania ubiegania się o taras
3. `REL_TARAS` - zwolnienie żądania ubiegania się miejsca na tarasie

4. REQ_WINDA_DOL - żądanie o miejsce w poczekalni na dole; zawiera priorytet żądania
5. ACK_WINDA_DOL - potwierdzenie żądania o poczekalnię na dole
6. REL_WINDA_DOL - zwolnienie poczekalni na dole (grupowa wiadomość)
7. REQ_WINDA_GORA - żądanie o miejsce w poczekalni na górze; zawiera priorytet żądania
8. ACK_WINDA_GORA - potwierdzenie żądania o poczekalnię na górze
9. REL_WINDA_GORA - zwolnienie poczekalni na górze (grupowa wiadomość)

4 Stany

1. NA_DOLE - proces odpoczywa na dole (stan początkowy)
2. CZEKA_NA_WINDE_DOL - ubiega się o poczekalnię na dole i czeka na zebranie grupy W
3. CZEKA_NA_WINDE_GORA - przebywa na tarasie
4. NA_TARASIE - ubiega się o poczekalnię na górze i czeka na zebranie grupy W

5 Obsługa zegara Lamporta

Przy każdym zdarzeniu:

- Wysyłanie wiadomości: `clock++`, dołącz `clock` do wiadomości
- Odbieranie wiadomości z timestampem `ts`: `clock = max(clock, ts) + 1`
- Zdarzenie lokalne (zmiana stanu): `clock++`

Priorytet żądania: mniejszy timestamp = wyższy priorytet. Przy równych timestampach: mniejszy PID = wyższy priorytet.

6 Algorytm

6.1 Skrócony opis algorytmu

Proces rozpoczyna ze stanem `NA_DOLE`. Po upływie losowej ilości czasu wysyła `REQ_TARAS` do wszystkich pozostałych procesów. Dopóki proces nie spełni odpowiednich wymagań, proces odpowiada na otrzymane wiadomości. Jeśli będąc w stanie `NA_DOLE` proces otrzyma `REQ_TARAS` i wartość licznika `ack_tars` nie osiągnął wartości `N-1` to proces zamiast odsyłać `ACK_TARAS` to dodaje proces do kolejki `queue_request` tylko po to, żeby po osiągnięciu `ack_tars == N-1` odesłać `ACK_TARAS` wszystkim procesom z `queue_request`.

Następnie proces po spełnieniu odpowiednich warunków ustawia licznik `ack_winda_dol` na 0, wysyła `REQ_WINDA_DOL` i zmienia stan na `CZEKA_NA_WINDE_DOL`. Proces podobnie reaguje na otrzymanie wiadomości `REQ_WINDA_DOL` jak na `REQ_TARAS` w stanie `NA_DOLE` tzn. dodaje procesy do `queue_request` dopóki nie `ack_winda_dol` nie osiągnie `N-1`. Po osiągnięciu wartości `N-1` przez licznik `ack_winda_dol`, proces oczywiście odsyła `ACK_WINDA_DOL` wszystkim procesom z `queue_request`. Dzięki wzajemnemu wykluczaniu się kolejek `queue_winda_dol` i `queue_winda_gora` **W**-ty proces kolejki `queue_winda_dol` swobodnie wysyła grupową wiadomość `REL_WINDA_DOL`. Pierwsze **W** procesów zmienia stan na `NA_TARASIE`.

Po upływie pewnego czasu proces wysyła `REQ_WINDA_GORA`, ustawia `ack_winda_gora` na 0 i zmienia stan. W ten sposób proces sygnalizuje, że chce zjechać windą na dół. Proces w stanie `CZEKA_NA_WINDA_GORA` reaguje na wiadomość `REQ_WINDA_GORA` analogicznie jak na `REQ_TARAS` w stanie `NA_DOLE`. Po spełnieniu warunków, **W** pierwszych procesów z kolejki `queue_winda_gora` zjeżdżają windą na dół, przy tym wysyłając `REL_WINDA_GORA` i `REL_TARAS`. Na koniec proces zmienia stan na `NA_DOLE`.

6.2 Szczegółowy opis algorytmu dla procesu *i*

1. Proces rozpoczyna ze stanem `NA_DOLE`.
2. Po losowym czasie odpoczynku proces ustawia `ack_tars = 0` i wysyła `REQ_TARAS` do wszystkich pozostałych procesów.

3. Aby osiągnąć stan CZEKA_NA_WINDE_DOL proces musi spełnić następujące warunki:
 - 3.1 $ack_taras == N-1$
 - 3.2 Pozycja procesu w kolejce `queue_taras` musi być mniejsza równa $\min(T-(T\%W), N-(N\%W))$
4. Po spełnieniu warunków z punktu 3. proces ustawia `ack_winda_dol = 0`, wysyła REQ_WINDA_DOL i zmienia stan na CZEKA_NA_WINDE_DOL.
5. Proces oczekuje aż $ack_winda_dol == N-1$, następnie sprawdza, czy jest W-tym procesem w kolejce `queue_winda_dol` oraz czy pierwszy proces w kolejce `queue_winda_dol` posiada większy priorytet niż pierwszy proces w kolejce `queue_winda_gora`.
 - 5.1 Jeśli tak, to proces wysyła REL_WINDA_DOL. Żądanie przybiera postać: $(timestamp, PID[])$, gdzie `PID[]` to tablica pierwszych W procesów z kolejki `queue_winda_dol`
 - 5.2 Jeśli nie, to proces czeka
6. Jeśli proces otrzyma REL_WINDA_DOL i w tablicy `PID[]` znajduje się PID procesu to proces zmienia stan na NA_TARASIE.
7. Po upływie pewnego czasu proces ustawia `ack_winda_gora = 0`, wysyła REQ_WINDA_GORA i zmienia stan na CZEKA_NA_WINDE_GORA.
8. Proces oczekuje aż $ack_winda_gora == N - 1$, następnie sprawdza, czy jest W-tym procesem kolejki `queue_winda_gora` oraz czy pierwszy proces w kolejce `queue_winda_gora` posiada większy priorytet niż pierwszy proces w kolejce `queue_winda_dol`.
 - 8.1 Jeśli tak, to proces wysyła REL_WINDA_GORA. Żądanie ma postać: $(timestamp, PID[])$, gdzie `PID[]` to tablica pierwszych W procesów kolejki `queue_winda_gora`
 - 8.2 Jeśli nie, to proces czeka
9. Jeśli proces otrzyma REL_WINDA_GORA i w tablicy `PID[]` znajduje się PID procesu to proces zmienia stan na NA_DOLE oraz wysyła REL_TARAS.

7 Obsługa wiadomości

7.1 Standardowa obsługa wiadomości

1. Niezależnie od stanu, poniżej znajdują się podstawowa reakcja na wiadomości dotyczące wszystkich sekcji. Dla sekcji $X \in \{\text{TARAS}, \text{WINDA_DOL}, \text{WINDA_GORA}\}$:
 - REQ_X: dodaj żądanie nadawcy do `queue_X`, odsyłaj ACK_X
 - REL_X: usuń żądanie nadawcy z `queue_X`
 - ACK_X: jeśli proces aktualnie ubiega się o X - zwiększ odpowiedni licznik; w przeciwnym razie ignoruj

7.2 Specyficzna obsługa wiadomości

1. NA_DOLE

- 1.1 REQ_TARAS - jeśli priorytet otrzymanego żądania jest mniejszy niż priorytet procesu i `ack_taras < N-1` to:

- Proces nie odsyła ACK_TARAS
- Dodaje proces do kolejki `queue_request`

Jeśli natomiast licznik `ack_taras` osiągnie wartość N-1 to proces odsyła wszystkim procesom z kolejki `queue_request` wiadomość ACK_TARAS

2. CZEKA_NA_WINDE_DOL

- 2.1 REQ_WINDA_DOL - jeśli priorytet otrzymanego żądania jest mniejszy niż priorytet procesu i `ack_winda_dol < N-1` to:

- Proces nie odsyła ACK_WINDA_DOL
- Dodaje proces do kolejki `queue_request`

Jeśli natomiast licznik `ack_winda_dol` osiągnie wartość N-1 to proces odsyła wszystkim procesom z kolejki `queue_request` wiadomość ACK_WINDA_DOL.

2.2 REL_WINDA_DOL

- Jeśli w tablicy `PID[]` jest PID procesu to zmienia stan na `NA_TARASIE`
- Należy sprawdzić warunki wejścia do windy (**6.2.5**)

3. CZEKA_NA_WINDE_GORA

3.1 REQ_WINDA_GORA - jeśli priorytet otrzymanego żądania jest mniejszy niż priorytet procesu i `ack_winda_gora < N-1` to:

- Proces nie odsyła `ACK_WINDA_GORA`
- Dodaje proces do kolejki `queue_request`

Jeśli natomiast licznik `ack_winda_gora` osiągnie wartość `N-1` to proces odsyła wszystkim procesom z kolejki `queue_request` wiadomość `ACK_WINDA_GORA`

3.2 REL_WINDA_DOL/REL_WINDA_GORA

- Jeśli w tablicy `PID[]` jest PID procesu to zmienia stan na `NA_DOLE` oraz wysyła `REL_TARAS`
- Należy sprawdzić warunki wejścia do windy (**6.2.8**)

8 Złożoność komunikacyjna

Zasób	REQ	ACK	REL	Razem
TARAS	$N - 1$	$N - 1$	$N - 1$	$3(N - 1)$
WINDA_DOL	$N - 1$	$N - 1$	$N - 1$	$3(N - 1)$
WINDA_GORA	$N - 1$	$N - 1$	$N - 1$	$3(N - 1)$
Łącznie				$9(N - 1)$